

VIRTUS-CC

Capacitação em Sistemas Embarcados

FreeRTOS e CMSIS-RTOS v2

Lista de Atividades Práticas

Prioridades, Semáforos e Mutex

Aluno: Antonni Guedes Alves de Moraes

Placa utilizada: STM32F407VET6

Repositório contendo projetos STM32CUBEIDE e relatórios individuais:

<https://github.com/VEEETOOORRR/Embarcados-RTOS>

Atividade 1 — Quem deve executar primeiro?

1. Objetivo

Investigar o efeito das prioridades e do bloqueio voluntário no escalonamento de três tarefas.

2. Diagrama

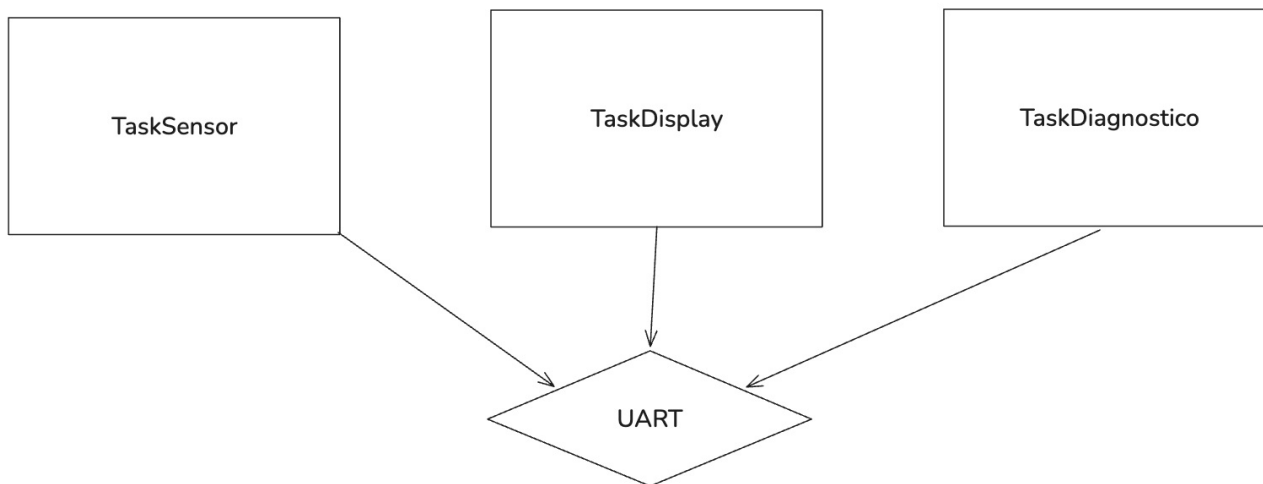


Diagrama da Atividade 1

3. Implementação e experimentos

Experimento A - Prioridades iguais

```
void TaskSensor_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        printf("[%lu ms] TaskSensor ON\r\n", osKernelGetTickCount());
    }
    /* USER CODE END 5 */
}
```

```
void TaskDisplay_fun(void *argument)
{
    /* USER CODE BEGIN TaskDisplay_fun */
    /* Infinite loop */
    for(;;)
    {
        printf("[%lu ms] TaskDisplay ON\r\n", osKernelGetTickCount());
    }
    /* USER CODE END TaskDisplay_fun */
}
```

```
void TaskDiagnostico_fun(void *argument)
{
    /* USER CODE BEGIN TaskDiagnostico_fun */
    /* Infinite loop */
    for(;;)
    {
        printf("[%lu ms] TaskDiagnostico ON\r\n", osKernelGetTickCount());
        //osDelay(2500);
    }
    /* USER CODE END TaskDiagnostico_fun */
}
```

Saída registrada:

```
TaskDiagnostico ON
TaskDiagnostico ON
TaskDiagnostico ON
TaskDiagnostico ON
TaskDiagnostico ON
...
```

```

[0 ms] TaskDisplay ONy ON
ON
[5 ms] TaskDisplay ON
6 ms] TaskDiagnostico ON
[[7 ms] TaskDisplay ON
ON
[[24 ms] TasDisplay ON
ON
[[30 ms] askDisplay ON
ON
[[36 m] TaskDisplay ON
ON
[[4 ms] TaskDisplay ON
ON
[[51 ms] Taskisplay ON
ON
[57 ms] TaskDisplay ON
ON
[63 ms] TaskDisplay ON
ON
[69 ms] TaskDisplay ON
ON
[75 ms] TaskDisplay ON
ON
[83 ms] TaskDisplay ON
6 ms] TaskDisplay ON
o ON
[95[96 ms] Taisplay ON
co ON
[[102 ms TaskDisplay ON
or ON[107 ms] TaskDisplay ON
ay ONens[116[117 maskDisplay ON
N
TaskD[125 ms] TaskDisplay ON
play ON
TaskDiagnostico ON
[137 ms] TaskDisplay ON
[138 ms] TaskDsplay ON
ON
[149 [150 mskDisplay ON
ON
ON
[158 ms] TaskDisplay ON
N
iaskSensor ON
[168 ms] TaskDisplay ON
[174 ms] TaskDisplay ON
askS[[180 s] TaskDisplay ON
81 ms[[88 ms] TaskDisplay ON
skDis[195 ms] TaskDisplay ON
Tas[[201 ms TaskDisplay ON
00 ms[207 ms] TaskDisplay ON

```

No trecho registrado, somente a mensagem de `TaskDiagnostico` aparece no terminal, porém a mesma se encontra “atropelada” como se a UART não conseguisse acompanhar o volume de mensagens a serem enviadas.

Experimento B - Prioridades diferentes

Saída registrada:

```

[0 ms] TaskSensor ON
[1 ms] TaskSensor ON
[3 ms] TaskSensor ON
[5 ms] TaskSensor ON
[7 ms] TaskSensor ON
[9 ms] TaskSensor ON
[11 ms] TaskSensor ON
[13 ms] TaskSensor ON
[15 ms] TaskSensor ON
[17 ms] TaskSensor ON
[19 ms] TaskSensor ON
[21 ms] TaskSensor ON
[23 ms] TaskSensor ON
[25 ms] TaskSensor ON
[27 ms] TaskSensor ON
[29 ms] TaskSensor ON
[31 ms] TaskSensor ON
[33 ms] TaskSensor ON
[35 ms] TaskSensor ON
[37 ms] TaskSensor ON
[39 ms] TaskSensor ON
[41 ms] TaskSensor ON
[43 ms] TaskSensor ON
[45 ms] TaskSensor ON
[47 ms] TaskSensor ON
[49 ms] TaskSensor ON
[51 ms] TaskSensor ON
[53 ms] TaskSensor ON
[55 ms] TaskSensor ON
[57 ms] TaskSensor ON
[59 ms] TaskSensor ON
[61 ms] TaskSensor ON
[63 ms] TaskSensor ON
[65 ms] TaskSensor ON
[67 ms] TaskSensor ON
[69 ms] TaskSensor ON

```

```

[71 ms] TaskSensor ON
[74 ms] TaskSensor ON
[76 ms] TaskSensor ON
[78 ms] TaskSensor ON
[80 ms] TaskSensor ON
[82 ms] TaskSensor ON
[84 ms] TaskSensor ON
[86 ms] TaskSensor ON
[88 ms] TaskSensor ON
[90 ms] TaskSensor ON
[92 ms] TaskSensor ON
[94 ms] TaskSensor ON
[96 ms] TaskSensor ON
[98 ms] TaskSensor ON
[100 ms] TaskSensor ON
[102 ms] TaskSensor ON
[104 ms] TaskSensor ON
[106 ms] TaskSensor ON
[108 ms] TaskSensor ON
[110 ms] TaskSensor ON
[112 ms] TaskSensor ON
[114 ms] TaskSensor ON
[116 ms] TaskSensor ON
[119 ms] TaskSensor ON
[121 ms] TaskSensor ON
[123 ms] TaskSensor ON
[125 ms] TaskSensor ON
...

```

No trecho registrado, somente a mensagem de `TaskSensor` aparece no terminal. A tarefa foi configurada com prioridade superior às demais.

Experimento C - Bloqueio voluntário

```

void TaskSensor_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        printf("[%lu ms] TaskSensor ON\r\n", osKernelGetTickCount());
        osDelay(1000);
    }
    /* USER CODE END 5 */
}

```

```

void TaskDisplay_fun(void *argument)
{
    /* USER CODE BEGIN TaskDisplay_fun */
    /* Infinite loop */
    for(;;)
    {
        printf("[%lu ms] TaskDisplay ON\r\n", osKernelGetTickCount());
        osDelay(1000);
    }
    /* USER CODE END TaskDisplay_fun */
}

```

```

void TaskDiagnostics_fun(void *argument)
{
    /* USER CODE BEGIN TaskDiagnostics_fun */
    /* Infinite loop */
    for(;;)
    {
        printf("[%lu ms] TaskDiagnostics ON\r\n", osKernelGetTickCount());
        osDelay(1000);
    }
    /* USER CODE END TaskDiagnostics_fun */
}

```

Prioridades iguais

Saída registrada:

```

[1001 ms] TaskDiagnostics[2002 ms] TaskSensor ON
[3003 ms] TaskDisplay ON
ON
[4004 ms] TaskDiagnostics[5005 ms] TaskSensor ON
[6006 ms] TaskDisplay ON
ON
[7007 ms] TaskDiagnostics[8008 ms] TaskSensor ON
[9009 ms] TaskDisplay ON
ON
[10010 ms] TaskDiagnostics[11011 ms] TaskSensor ON
[12012 ms] TaskDisplay ON
ON
[13013 ms] TaskDiagnostics[14014 ms] TaskSensor ON
[15015 ms] TaskDisplay ON
ON

```

```
[16016 ms] TaskDiagnostico[17017 ms] TaskSensor ON
[18018 ms] TaskDisplay ON
ON
[19019 ms] TaskDiagnostico[20020 ms] TaskSensor ON
[21021 ms] TaskDisplay ON
ON
[22022 ms] TaskDiagnostico[23023 ms] TaskSensor ON
[24024 ms] TaskDisplay ON
ON
[25025 ms] TaskDiagnostico[26026 ms] TaskSensor ON
[27027 ms] TaskDisplay ON
ON
[28028 ms] TaskDiagnostico[29029 ms] TaskSensor ON
[30030 ms] TaskDisplay ON
ON
[31031 ms] TaskDiagnostico[32032 ms] TaskSensor ON
[33033 ms] TaskDisplay ON
ON
[34034 ms] TaskDiagnostico[35035 ms] TaskSensor ON
[36036 ms] TaskDisplay ON
ON
[37037 ms] TaskDiagnostico[38038 ms] TaskSensor ON
[39039 ms] TaskDisplay ON
ON
[40040 ms] TaskDiagnostico[41041 ms] TaskSensor ON
[42042 ms] TaskDisplay ON
ON
```

O trecho mostra a sequência `TaskSensor → TaskDisplay → TaskDiagnostico`. Porém, a mensagem aparece “atropelada”. É evidente que somente uma task tem acesso à UART a cada intervalo de 1 segundo.

Prioridades diferentes

Saída registrada:

```
[0 ms] TaskSensor ON
[1 ms] TaskDisplay ON
[3 ms] TaskDiagnostico ON
[1001 ms] TaskSensor ON
[1003 ms] TaskDisplay ON
[1006 ms] TaskDiagnostico ON
[2003 ms] TaskSensor ON
[2005 ms] TaskDisplay ON
[2008 ms] TaskDiagnostico ON
[3005 ms] TaskSensor ON
[3007 ms] TaskDisplay ON
[3010 ms] TaskDiagnostico ON
[4007 ms] TaskSensor ON
[4009 ms] TaskDisplay ON
[4012 ms] TaskDiagnostico ON
[5009 ms] TaskSensor ON
[5011 ms] TaskDisplay ON
[5014 ms] TaskDiagnostico ON
[6011 ms] TaskSensor ON
[6013 ms] TaskDisplay ON
[6016 ms] TaskDiagnostico ON
[7013 ms] TaskSensor ON
[7015 ms] TaskDisplay ON
[7018 ms] TaskDiagnostico ON
[8015 ms] TaskSensor ON
[8017 ms] TaskDisplay ON
[8020 ms] TaskDiagnostico ON
...
```

O trecho mostra grupos de mensagens na ordem apresentada. A cada intervalo de 1 segundo, as 3 tasks fazem uso da UART, transmitindo as mensagens na sequência `TaskSensor → TaskDisplay → TaskDiagnostico`.

Seguindo prioridades e valores de osDelay distintos

- `TaskSensor -> osDelay(300), osPriorityHigh`
- `TaskDisplay -> osDelay(1000), osPriorityNormal`
- `TaskDiagnostico -> osDelay(2500), osPriorityLow`

Saída registrada:

```
[0 ms] TaskSensor ON
[1 ms] TaskDisplay ON
[3 ms] TaskDiagnostico ON
[301 ms] TaskSensor ON
[603 ms] TaskSensor ON
[905 ms] TaskSensor ON
[1003 ms] TaskDisplay ON
[1207 ms] TaskSensor ON
[1509 ms] TaskSensor ON
[1811 ms] TaskSensor ON
[2005 ms] TaskDisplay ON
[2113 ms] TaskSensor ON
[2415 ms] TaskSensor ON
[2506 ms] TaskDiagnostico ON
[2717 ms] TaskSensor ON
```

```
[3007 ms] TaskDisplay ON
[3019 ms] TaskSensor ON
[3321 ms] TaskSensor ON
[3623 ms] TaskSensor ON
[3925 ms] TaskSensor ON
[4009 ms] TaskDisplay ON
[4227 ms] TaskSensor ON
[4529 ms] TaskSensor ON
[4831 ms] TaskSensor ON
[5008 ms] TaskDiagnostico ON
[5011 ms] TaskDisplay ON
[5133 ms] TaskSensor ON
[5435 ms] TaskSensor ON
[5737 ms] TaskSensor ON
```

5. Análise dos resultados

Questão: Aumentar a prioridade de uma tarefa significa necessariamente que ela executará mais vezes? Explique utilizando os estados Running, Ready e Blocked.

Não necessariamente. A prioridade influencia a seleção de uma tarefa que está em **Ready** para ocupar a CPU e entrar em **Running**. Uma tarefa de prioridade alta pode executar menos vezes que outra de prioridade baixa se permanecer por mais tempo em **Blocked**, aguardando um atraso ou a disponibilidade de um recurso. Por isso, a quantidade de execuções também depende do comportamento de cada tarefa.

Atividade 2 — Atendimento de uma situação crítica

1. Objetivo

Investigar a influência da prioridade de uma tarefa de emergência e da carga de CPU sobre seu atendimento.

2. Diagrama

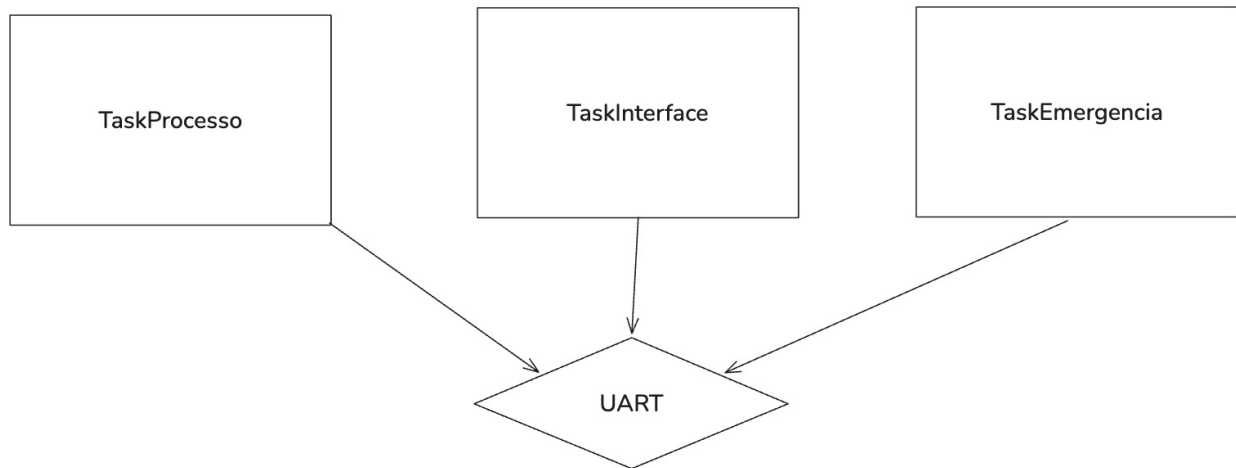


Diagrama da Atividade 2

3. Implementação e experimentos

Para simular uma task de uso intensivo de CPU, foi implementado um laço for de contagem até 50 milhões.

```
void TaskProcesso_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    volatile uint32_t contador = 0;
    for(;;)
    {
        for(uint32_t i = 0; i < 50000000; i++){
            contador++;
        }
        uint32_t tick_atual = osKernelGetTickCount();
        printf("[%lu ms] TaskProcesso: Encerrado ciclo de contagem\r\n", tick_atual);
    }
    /* USER CODE END 5 */
}
```

```
void TaskInterface_fun(void *argument)
{
    /* USER CODE BEGIN TaskInterface_fun */
    /* Infinite loop */
    for(;;)
    {
        uint32_t tick_atual = osKernelGetTickCount();
        printf("[%lu ms] TaskInterface: Atualizando interface!\r\n", tick_atual);
        osDelay(1000);
    }
    /* USER CODE END TaskInterface_fun */
}
```

```
void TaskEmergencia_fun(void *argument)
{
    /* USER CODE BEGIN TaskEmergencia_fun */
    /* Infinite loop */
    for(;;)
    {
        uint32_t tick_atual = osKernelGetTickCount();
        printf("[%lu ms] TaskEmergencia: Emergencia simulada!\r\n", tick_atual);
        osDelay(2500);
    }
    /* USER CODE END TaskEmergencia_fun */
}
```

Mesma prioridade

```
[1002 ms] TaskInterface: Atualizando interface!  
[2008 ms] TaskInterface: Atualizando interface!  
[2506 ms] TaskEmergencia: Emergencia simulada!  
[3014 ms] TaskInterface: Atualizando interface!  
[4020 ms] TaskInterface: Atualizando interface!  
[5012 ms] TaskEmergencia: Emergencia simulada!  
[5026 ms] TaskInterface: Atualizando interface!  
[5105 ms] TaskProcessso: Encerrado ciclo de contagem  
[6032 ms] TaskInterface: Atualizando interface!  
[7038 ms] TaskInterface: Atualizando interface!  
[7518 ms] TaskEmergencia: Emergencia simulada!  
[8044 ms] TaskInterface: Atualizando interface!
```

TaskEmergencia com prioridade LOW

```
[1006 ms] TaskInterface: Atualizando interface!  
[2012 ms] TaskInterface: Atualizando interface!  
[3018 ms] TaskInterface: Atualizando interface!  
[4024 ms] TaskInterface: Atualizando interface!  
[5030 ms] TaskInterface: Atualizando interface!  
[5097 ms] TaskProcessso: Encerrado ciclo de contagem  
[6036 ms] TaskInterface: Atualizando interface!  
[7042 ms] TaskInterface: Atualizando interface!  
[8048 ms] TaskInterface: Atualizando interface!  
[9054 ms] TaskInterface: Atualizando interface!  
[10060 ms] TaskInterface: Atualizando interface!  
[10196 ms] TaskProcessso: Encerrado ciclo de contagem
```

Nenhuma mensagem de `TaskEmergencia` aparece no trecho registrado para prioridade LOW.

TaskEmergencia com prioridade HIGH

```
[1010 ms] TaskInterface: Atualizando interface!  
[2016 ms] TaskInterface: Atualizando interface!  
[2503 ms] TaskEmergencia: Emergencia simulada!  
[3022 ms] TaskInterface: Atualizando interface!  
[4028 ms] TaskInterface: Atualizando interface!  
[5007 ms] TaskEmergencia: Emergencia simulada!  
[5034 ms] TaskInterface: Atualizando interface!  
[5110 ms] TaskProcessso: Encerrado ciclo de contagem  
[6040 ms] TaskInterface: Atualizando interface!  
[7046 ms] TaskInterface: Atualizando interface!  
[7511 ms] TaskEmergencia: Emergencia simulada!  
[8052 ms] TaskInterface: Atualizando interface!
```

É possível observar uma diferença sutil no tempo de resposta da `TaskEmergencia` quando esta possui prioridade maior, comparado com quando possui prioridade igual às outras tasks.

Mesma prioridade: 2506ms entre execuções.

Prioridade maior: 2503ms entre execuções.

4. Análise dos resultados

Questão: Uma tarefa possuir prioridade elevada é suficiente para garantir que ela sempre apresentará um pequeno tempo de resposta? Justifique.

Não necessariamente. Caso uma task de alta prioridade esteja bloqueada aguardando a liberação de um recurso crítico ou em ready esperando a CPU em um SO não preemptivo, não é possível garantir sua execução rapidamente.

Atividade 3 — Só processe quando houver dados

1. Objetivo

Comparar polling e sincronização por semáforo binário na produção e no processamento de dados.

2. Diagrama

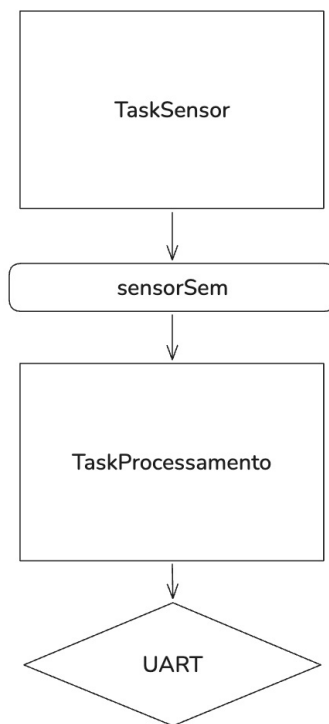


Diagrama da Atividade 3

3. Implementação e experimentos

`TaskSensor` lê o estado de do botão K1 embutido na placa STM32F407VET6, soldado ao pino PE3. Para transmitir o estado desse botão, foi utilizada uma fila de comprimento 1. `TaskSensor` deposita a leitura do botão, e `TaskProcessamento` recebe o valor, conforme os nomes empregados no código.

Solução de polling

```
void TaskSensor_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    uint8_t valor_sensor = 0;
    /* Infinite loop */
    for(;;)
    {
        if(HAL_GPIO_ReadPin(BTN_K1_GPIO_Port, BTN_K1_Pin)){
            valor_sensor = 1;
        }
        else valor_sensor = 0;
        osMessageQueuePut(filaSensorHandle, &valor_sensor, 0, 0);
        uint32_t tick_atual = osKernelGetTickCount();
        printf("[%lu ms] TaskSensor: Enviado valor %d\r\n", tick_atual, valor_sensor);
        osDelay(100);
    }
    /* USER CODE END 5 */
}
```

```

void TaskProcessamento_fun(void *argument)
{
    /* USER CODE BEGIN TaskProcessamento_fun */
    uint8_t valor_recebido = 0;
    /* Infinite loop */
    for(;;)
    {
        if(osMessageQueueGetCount(filaSensorHandle) != 0) {
            osMessageQueueGet(filaSensorHandle, &valor_recebido, NULL, 0);
            uint32_t tick_atual = osKernelGetTickCount();
            printf("[%lu ms] TaskProcessamento: Recebido valor: %d\r\n", tick_atual, valor_recebido);
        }
        else {
            uint32_t tick_atual = osKernelGetTickCount();
            printf("[%lu ms] TaskProcessamento: Nenhum valor recebido!\r\n", tick_atual);
        }
    }
    /* USER CODE END TaskProcessamento_fun */
}

```

Saída registrada:

```

[500 ms] TaskSensor: Enviado valor 1
or recebido!
[504 ms] TaskProcessamento: Recebido valor: 1
[508 ms] TaskProcessamento: Nenhum valor recebido!
[513 ms] TaskProcessamento: Nenhum valor recebido!
[518 ms] TaskProcessamento: Nenhum valor recebido!
[522 ms] TaskProcessamento: Nenhum valor recebido!
[527 ms] TaskProcessamento: Nenhum valor recebido!
[531 ms] TaskProcessamento: Nenhum valor recebido!
[536 ms] TaskProcessamento: Nenhum valor recebido!
[540 ms] TaskProcessamento: Nenhum valor recebido!
[545 ms] TaskProcessamento: Nenhum valor recebido!
[549 ms] TaskProcessamento: Nenhum valor recebido!
[554 ms] TaskProcessamento: Nenhum valor recebido!
[558 ms] TaskProcessamento: Nenhum valor recebido!
[563 ms] TaskProcessamento: Nenhum valor recebido!
[568 ms] TaskProcessamento: Nenhum valor recebido!
[572 ms] TaskProcessamento: Nenhum valor recebido!
[577 ms] TaskProcessamento: Nenhum valor recebido!
[581 ms] TaskProcessamento: Nenhum valor recebido!
[586 ms] TaskProcessamento: Nenhum valor recebido!
[590 ms] TaskProcessamento: Nenhum valor recebido!
[595 ms] TaskProcessamento: Nenhum valor recebido!
[599 ms] TaskSensor: Enviado valor 1
or recebido!
[604 ms] TaskProcessamento: Recebido valor: 1
[608 ms] TaskProcessamento: Nenhum valor recebido!
[613 ms] TaskProcessamento: Nenhum valor recebido!

```

A tarefa `TaskProcessamento` desperdiça tempo de CPU verificando se a tarefa `TaskSensor` gerou um novo valor.

Versão com o semáforo `sensorSem`

```

void TaskSensor_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    uint8_t valor_sensor = 0;
    /* Infinite loop */
    for(;;)
    {
        if(HAL_GPIO_ReadPin(BTN_K1_GPIO_Port, BTN_K1_Pin)){
            valor_sensor = 1;
        }
        else valor_sensor = 0;
        osMessageQueuePut(filaSensorHandle, &valor_sensor, 0, 0);
        uint32_t tick_atual = osKernelGetTickCount();
        printf("[%lu ms] TaskSensor: Enviado valor %d\r\n", tick_atual, valor_sensor);
        osSemaphoreRelease(sensorSemHandle);
        osDelay(100);
    }
    /* USER CODE END 5 */
}

```

```

void TaskProcessamento_fun(void *argument)
{
    /* USER CODE BEGIN TaskProcessamento_fun */
    uint8_t valor_recebido = 0;
    /* Infinite loop */
    for(;;)
    {
        osMessageQueueGet(filaSensorHandle, &valor_recebido, NULL, 0);
        uint32_t tick_atual = osKernelGetTickCount();
        printf("[%lu ms] TaskProcessamento: Recebido valor: %d\r\n", tick_atual, valor_recebido);
        osSemaphoreAcquire(sensorSemHandle, osWaitForever);
    }
    /* USER CODE END TaskProcessamento_fun */
}

```

Saída registrada:

```
...
[3191 ms] TaskSensor: Enviado valor 1
[3194 ms] TaskProcessamento: Recebido valor: 1
[3294 ms] TaskSensor: Enviado valor 1
[3297 ms] TaskProcessamento: Recebido valor: 1
[3397 ms] TaskSensor: Enviado valor 1
[3400 ms] TaskProcessamento: Recebido valor: 1
[3500 ms] TaskSensor: Enviado valor 1
[3503 ms] TaskProcessamento: Recebido valor: 1
[3603 ms] TaskSensor: Enviado valor 0
[3606 ms] TaskProcessamento: Recebido valor: 0
[3706 ms] TaskSensor: Enviado valor 0
```

4. Análise dos resultados

Questão: Compare as soluções com polling e semáforo. Qual delas utiliza melhor os recursos do sistema operacional? Explique.

No registro de polling, a tarefa de processamento realiza verificações repetidas e emite mensagens quando não há dados. No registro com semáforo, as mensagens de processamento acompanham as mensagens do produtor. Dessa forma, não consome processamento do SO desnecessariamente.

Atividade 4 — Recursos limitados

1. Objetivo

Representar a disponibilidade de vagas de estacionamento com um semáforo contador.

2. Diagrama

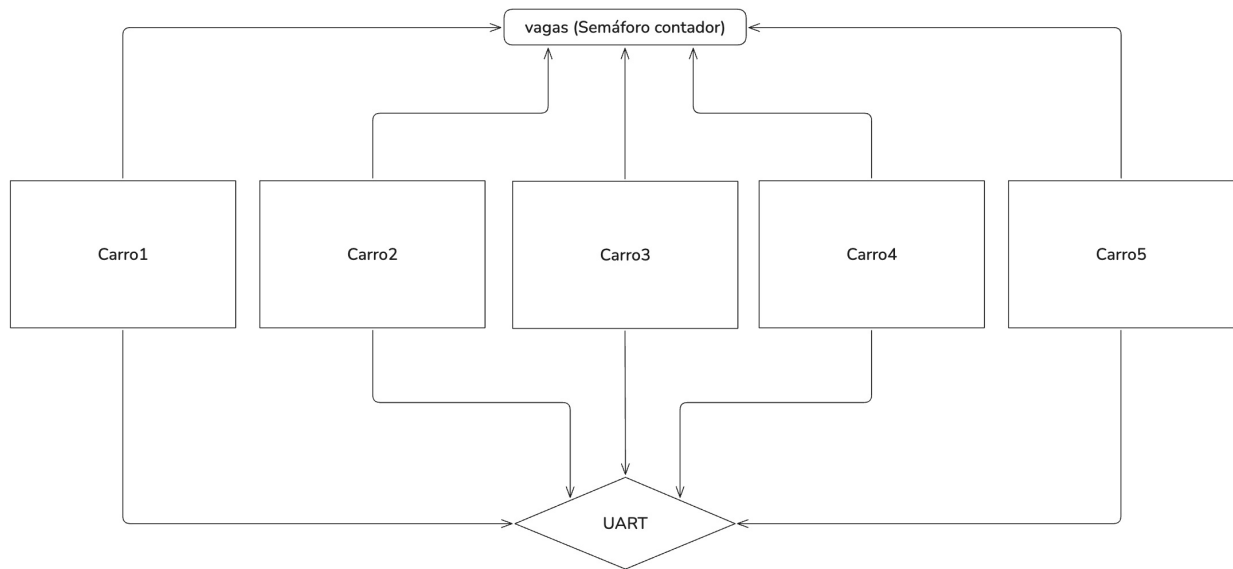


Diagrama da Atividade 4

3. Implementação e experimentos

```
void Carro1_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    osDelay(100 + (rand() % 100));
    uint32_t tempo_estacionado;
    uint32_t tempo_passeando;
    for(;;)
    {
        printf("[%lu ms] Carro 1: Tentando entrar...\r\n", osKernelGetTickCount());
        osSemaphoreAcquire(vagasHandle, osWaitForever);
        printf("[%lu ms] Carro 1: Entrada autorizada.\r\n", osKernelGetTickCount());
        // Simula tempo estacionado aleatório (entre 1000ms e 3000ms)
        tempo_estacionado = 1000 + (rand() % 2001);
        osDelay(tempo_estacionado);
        printf("[%lu ms] Carro 1: Saindo após %lu ms estacionado.\r\n", osKernelGetTickCount(), tempo_estacionado);
        osSemaphoreRelease(vagasHandle);
        // Simula tempo passeando pela cidade antes de tentar estacionar de novo (entre 2000ms e 5000ms)
        tempo_passeando = 2000 + (rand() % 3001);
        osDelay(tempo_passeando);
    }
    /* USER CODE END 5 */
}
```

Os 5 carros seguem o modelo acima.

OBS: A saída do terminal apresenta problemas de mensagens faltantes, devido ao uso concorrente da UART. Um mutex pode ser implementado posteriormente para garantir a integridade do envio das mensagens.

3 vagas

```
[100 ms] Carro 5: Tentando entrar...
[103 ms] Carro 5: Entrada autorizada.
[129 ms] Carro 4: Tentando entrar...
[132 ms] Carro 1: Entrada autorizada.
[143 ms] Carro 2: Tentando entrar...
[162 ms] Carro 3: Tentando entrar...
[2042 ms] Carro 1: Saindo após 1909 ms estacionado.
[2046 ms] Carro 2: Entrada autorizada.
[2324 ms] Carro 4: Saindo após 2189 ms estacionado.
[2328 ms] Carro 3: Entrada autorizada.
[2675 ms] Carro 5: Saindo após 2569 ms estacionado.
```

```
[3280 ms] Carro 2: Saindo após 1230 ms estacionado.
[4705 ms] Carro 5: Tentando entrar...
[4708 ms] Carro 5: Entrada autorizada.
[5156 ms] Carro 1: Tentando entrar...
[5159 ms] Carro 1: Entrada autorizada.
[5231 ms] Carro 3: Saindo após 2899 ms estacionado.
[5596 ms] Carro 4: Tentando entrar...
[5599 ms] Carro 4: Entrada autorizada.
[6413 ms] Carro 2: Tentando entrar...
[6860 ms] Carro 4: Saindo após 1258 ms estacionado.
[6864 ms] Carro 2: Entrada autorizada.
[7549 ms] Carro 5: Saindo após 2838 ms estacionado.
[7920 ms] Carro 1: Saindo após 2758 ms estacionado.
[8566 ms] Carro 2: Saindo após 1698 ms estacionado.
[8800 ms] Carro 3: Tentando entrar...
[8803 ms] Carro 3: Entrada autorizada.
[10287 ms] Carro 5: Tentando entrar...
[10290 ms] Carro 5: Entrada autorizada.
[10681 ms] Carro 3: Saindo após 1875 ms estacionado.
```

2 vagas

```
[100 ms] Carro 5: Tentando entrar...
[103 ms] Carro 5: Entrada autorizada.
[129 ms] Carro 4: Tentando entrar...
[132 ms] Carro 1: Tentando entrar...
[143 ms] Carro 2: Tentando entrar...
[162 ms] Carro 3: Tentando entrar...
[2044 ms] Carro 4: Saindo após 1909 ms estacionado.
[2048 ms] Carro 1: Entrada autorizada.
[2675 ms] Carro 5: Saindo após 2569 ms estacionado.
[2679 ms] Carro 2: Entrada autorizada.
[4176 ms] Carro 2: Saindo após 1493 ms estacionado.
[4180 ms] Carro 3: Entrada autorizada.
[4201 ms] Carro 1: Saindo após 2149 ms estacionado.
[5400 ms] Carro 4: Tentando entrar...
[5403 ms] Carro 4: Entrada autorizada.
[5544 ms] Carro 5: Tentando entrar...
[5621 ms] Carro 3: Saindo após 1437 ms estacionado.
[5625 ms] Carro 5: Entrada autorizada.
```

1 vaga

```
[100 ms] Carro 5: Tentando entrar...
[103 ms] Carro 5: Entrada autorizada.
[129 ms] Carro 4: Tentando entrar...
[133 ms] Carro 1: Tentando entrar...
[143 ms] Carro 2: Tentando entrar...
[162 ms] Carro 3: Tentando entrar...
[2675 ms] Carro 5: Saindo após 2569 ms estacionado.
[2679 ms] Carro 4: Entrada autorizada.
[4872 ms] Carro 4: Saindo após 2189 ms estacionado.
[4876 ms] Carro 1: Entrada autorizada.
[5757 ms] Carro 5: Tentando entrar...
[6110 ms] Carro 1: Saindo após 1230 ms estacionado.
[6114 ms] Carro 2: Entrada autorizada.
[7986 ms] Carro 4: Tentando entrar...
[9017 ms] Carro 2: Saindo após 2899 ms estacionado.
[9021 ms] Carro 3: Entrada autorizada.
[9382 ms] Carro 1: Tentando entrar...
[10454 ms] Carro 3: Saindo após 1429 ms estacionado.
[10458 ms] Carro 5: Entrada autorizada.
[11047 ms] Carro 2: Tentando entrar...
[13220 ms] Carro 5: Saindo após 2758 ms estacionado.
[13224 ms] Carro 4: Entrada autorizada.
[14486 ms] Carro 4: Saindo após 1258 ms estacionado.
[14490 ms] Carro 1: Entrada autorizada.
[14547 ms] Carro 3: Tentando entrar...
[16192 ms] Carro 1: Saindo após 1698 ms estacionado.
[16196 ms] Carro 2: Entrada autorizada.
[16789 ms] Carro 5: Tentando entrar...
[17972 ms] Carro 2: Saindo após 1772 ms estacionado.
[17976 ms] Carro 3: Entrada autorizada.
[18502 ms] Carro 4: Tentando entrar...
[18930 ms] Carro 1: Tentando entrar...
[19855 ms] Carro 3: Saindo após 1875 ms estacionado.
[19859 ms] Carro 5: Entrada autorizada.
[20346 ms] Carro 2: Tentando entrar...
[21434 ms] Carro 5: Saindo após 1571 ms estacionado.
[21438 ms] Carro 4: Entrada autorizada.
[23633 ms] Carro 4: Saindo após 2191 ms estacionado.
```

4. Análise dos resultados

Questão: Quando um semáforo contador possui apenas uma unidade disponível, qual é o comportamento observado? Ele se torna equivalente a um mutex? Discuta as diferenças conceituais.

Funcionalmente, possui um comportamento semelhante a um mutex, onde apenas um carro tem acesso ao recurso compartilhado de cada vez. Porém, ao contrário de um mutex, outros carros além do que está ocupado a vaga podem dar um “release”.

Atividade 5 — Compartilhamento da UART

1. Objetivo

Comparar o compartilhamento da UART sem proteção e com proteção por mutex.

2. Diagrama

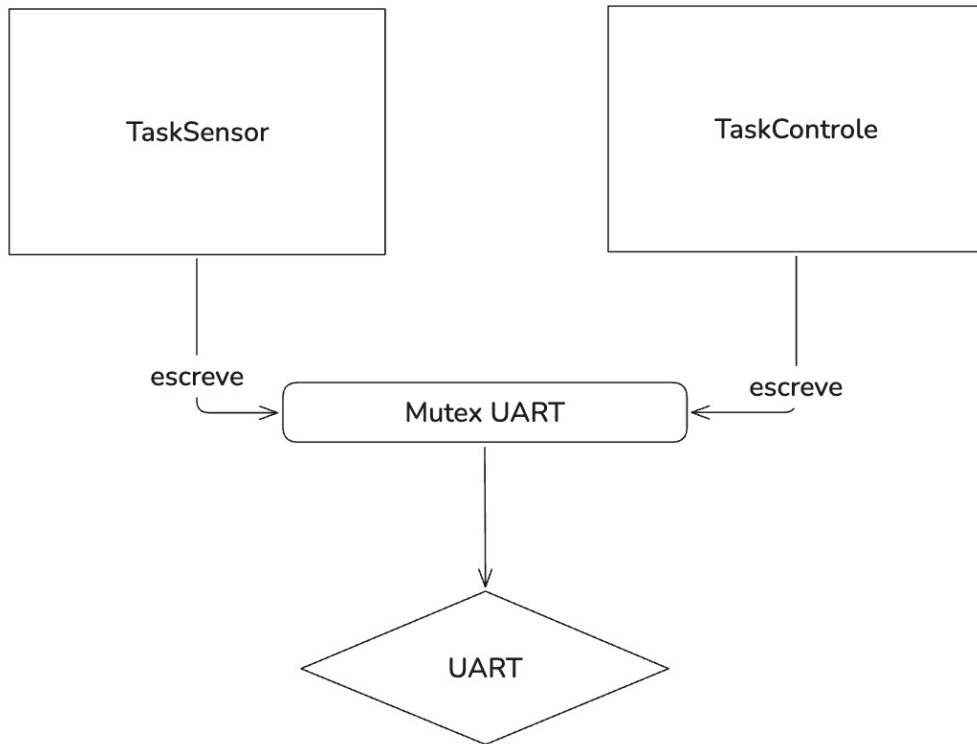


Diagrama da Atividade 5

3. Implementação e experimentos

Sem uso de mutex

```
void TaskSensor_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        for(int i = 0; i < 50; i++){
            printf("TASKSENSOR\r\n");
        }
    }
    /* USER CODE END 5 */
}
```

```
void TaskControle_fun(void *argument)
{
    /* USER CODE BEGIN TaskControle_fun */
    /* Infinite loop */
    for(;;)
    {
        for(int i = 0; i < 50; i++){
            printf("TASKCONTROLE\r\n");
        }
    }
    /* USER CODE END TaskControle_fun */
}
```

Saída registrada:

```
TASKSENSOR
TASKCONTROLE
```

```
TASKSETAOR
NTROLE
TASKCONTROLE
TASKSENSOR
TASKCONTROLE
TASKSETTASKCONTROLE
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKCONTROLE
```

Sem o uso do mutex, o terminal apresenta mensagens fragmentadas e sobrepostas.

Com uso de mutex

```
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
TASKCONTROLE
...
TASKCONTROLE
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
TASKSENSOR
...
```

Fazendo uso do mutex, a cada task imprime sua mensagem via UART 50 vezes sem “atropelo”.

4. Análise dos resultados

Questão: Por que um mutex é conceitualmente mais adequado para proteger a UART do que um semáforo?

Fazendo uso de um mutex, a tarefa que o adquire é responsável por liberá-lo. Na UART, a proteção deve abranger toda a região de transmissão que precisa permanecer íntegra, e todos os acessos concorrentes devem respeitar o mesmo mecanismo.

Atividade 6 — O contador que apresenta o valor errado

1. Objetivo

Investigar o resultado de incrementos concorrentes em uma variável compartilhada e a proteção com mutex.

2. Diagrama

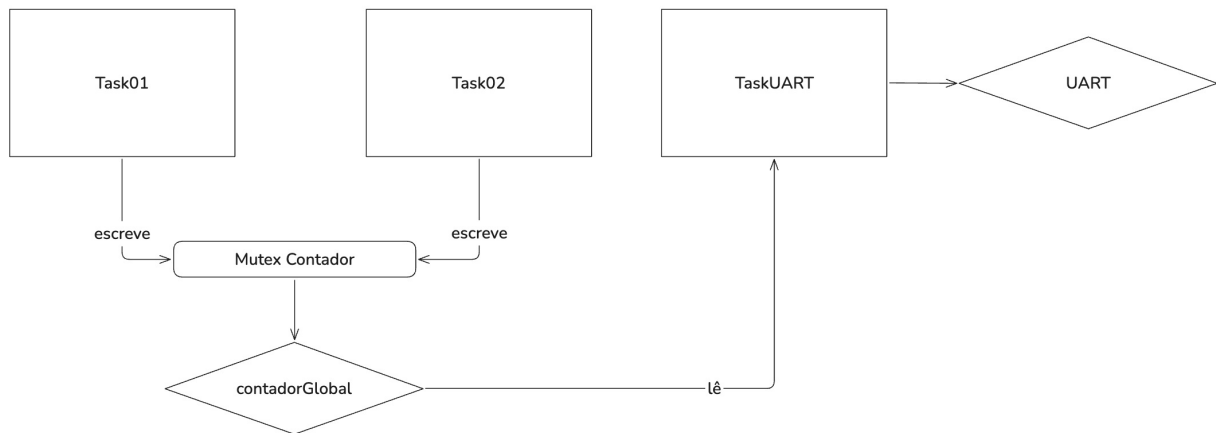


Diagrama da Atividade 6

3. Implementação e experimentos

Sem usar mutex

```
void Task01_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    for(int i = 0; i < 100000; i++){
        contadorGlobal++;
    }
    /* Infinite loop */
    for(;;)
    {
        osDelay(1);
    }
    /* USER CODE END 5 */
}
```

```
void Task02_fun(void *argument)
{
    /* USER CODE BEGIN Task02_fun */
    for(int i = 0; i < 100000; i++){
        contadorGlobal++;
    }
    /* Infinite loop */
    for(;;)
    {
        osDelay(1);
    }
    /* USER CODE END Task02_fun */
}
```

```
void TaskUART_fun(void *argument)
{
    /* USER CODE BEGIN TaskUART_fun */
    osDelay(1000);
    printf("%lu\r\n", contadorGlobal);
    /* Infinite loop */
    for(;;)
    {
        osDelay(1);
    }
    /* USER CODE END TaskUART_fun */
}
```

Saída registrada:

163725

Proteção com mutex

```
void Task01_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    for(int i = 0; i < 100000; i++){
        osMutexAcquire(contadorMutexHandle, osWaitForever);
        contadorGlobal++;
        osMutexRelease(contadorMutexHandle);
    }
    /* Infinite loop */
    for(;;)
    {
        osDelay(1);
    }
    /* USER CODE END 5 */
}
```

```
void Task02_fun(void *argument)
{
    /* USER CODE BEGIN Task02_fun */
    for(int i = 0; i < 100000; i++){
        osMutexAcquire(contadorMutexHandle, osWaitForever);
        contadorGlobal++;
        osMutexRelease(contadorMutexHandle);
    }
    /* Infinite loop */
    for(;;)
    {
        osDelay(1);
    }
    /* USER CODE END Task02_fun */
}
```

Saída registrada:

200000

4. Análise dos resultados

Questão: Explique por que `contadorGlobal++` não deve ser considerado necessariamente atômico. Utilize a sequência LER → MODIFICAR → ESCREVER.

O incremento pode envolver três etapas: **ler** o valor da memória para um registrador, **modificar** esse valor e **escrever** o resultado na memória. Se outra tarefa acessar a variável entre essas etapas, um incremento pode sobrescrever o resultado de outro.

Atividade 7 — Se estiver ocupado, faça outra coisa

1. Objetivo

Comparar a espera indefinida, a tentativa imediata e a espera limitada para adquirir um recurso.

2. Diagrama

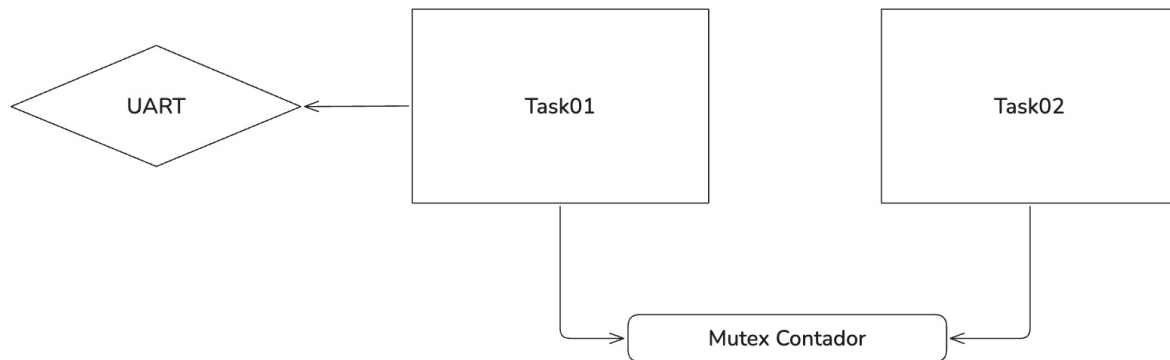


Diagrama da Atividade 7

3. Implementação e experimentos

```
void Task01_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        if(osMutexAcquire(recursoMutexHandle, osWaitForever) == osOK){
            printf("[%Lu ms] Mutex adquirido. Fazendo uso do recurso compartilhado!\r\n", osKernelGetTickCount());
            osMutexRelease(recursoMutexHandle);
        }
        else {
            printf("[%Lu ms] Mutex ocupado. Fazendo outra coisa...\r\n", osKernelGetTickCount());
        }
        osDelay(200);
    }
    /* USER CODE END 5 */
}
```

```
void Task02_fun(void *argument)
{
    /* USER CODE BEGIN Task02_fun */
    /* Infinite loop */
    for(;;)
    {
        osMutexAcquire(recursoMutexHandle, osWaitForever);
        osDelay(30);
        osMutexRelease(recursoMutexHandle);
        osDelay(5);
    }
    /* USER CODE END Task02_fun */
}
```

saída usando osWaitForever

```
[30 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[240 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[450 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[660 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[870 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[1080 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[1290 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[1500 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[1710 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[1920 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[2130 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[2340 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
[2550 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
```

```
[2760 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[2970 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[3180 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[3390 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[3600 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[3810 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[4020 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[4230 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[4440 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[4650 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[4860 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[5070 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[5280 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
```

saída usando timeout = 0

```
[0 ms] Mutex ocupado. Fazendo outra coisa...  
[204 ms] Mutex ocupado. Fazendo outra coisa...  
[408 ms] Mutex ocupado. Fazendo outra coisa...  
[612 ms] Mutex ocupado. Fazendo outra coisa...  
[816 ms] Mutex ocupado. Fazendo outra coisa...  
[1020 ms] Mutex ocupado. Fazendo outra coisa...  
[1224 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[1429 ms] Mutex ocupado. Fazendo outra coisa...  
[1633 ms] Mutex ocupado. Fazendo outra coisa...  
[1837 ms] Mutex ocupado. Fazendo outra coisa...  
[2041 ms] Mutex ocupado. Fazendo outra coisa...  
[2245 ms] Mutex ocupado. Fazendo outra coisa...  
[2449 ms] Mutex ocupado. Fazendo outra coisa...  
[2653 ms] Mutex ocupado. Fazendo outra coisa...  
[2857 ms] Mutex ocupado. Fazendo outra coisa...  
[3061 ms] Mutex ocupado. Fazendo outra coisa...  
[3265 ms] Mutex ocupado. Fazendo outra coisa...  
[3469 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[3674 ms] Mutex ocupado. Fazendo outra coisa...  
[3878 ms] Mutex ocupado. Fazendo outra coisa...  
[4082 ms] Mutex ocupado. Fazendo outra coisa...  
[4286 ms] Mutex ocupado. Fazendo outra coisa...  
[4490 ms] Mutex ocupado. Fazendo outra coisa...  
[4694 ms] Mutex ocupado. Fazendo outra coisa...  
[4898 ms] Mutex ocupado. Fazendo outra coisa...  
[5102 ms] Mutex ocupado. Fazendo outra coisa...  
[5306 ms] Mutex ocupado. Fazendo outra coisa...  
[5510 ms] Mutex ocupado. Fazendo outra coisa...  
[5714 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
```

Utilizando timeout = 10

```
[0 ms] Mutex ocupado. Fazendo outra coisa...  
[10 ms] Mutex ocupado. Fazendo outra coisa...  
[224 ms] Mutex ocupado. Fazendo outra coisa...  
[438 ms] Mutex ocupado. Fazendo outra coisa...  
[652 ms] Mutex ocupado. Fazendo outra coisa...  
[866 ms] Mutex ocupado. Fazendo outra coisa...  
[1080 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[1290 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[1500 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[1710 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[1920 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[2130 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[2340 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[2550 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[2760 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
```

Utilizando timeout = 7

```
[7 ms] Mutex ocupado. Fazendo outra coisa...  
[218 ms] Mutex ocupado. Fazendo outra coisa...  
[429 ms] Mutex ocupado. Fazendo outra coisa...  
[640 ms] Mutex ocupado. Fazendo outra coisa...  
[851 ms] Mutex ocupado. Fazendo outra coisa...  
[1062 ms] Mutex ocupado. Fazendo outra coisa...  
[1273 ms] Mutex ocupado. Fazendo outra coisa...  
[1484 ms] Mutex ocupado. Fazendo outra coisa...  
[1695 ms] Mutex ocupado. Fazendo outra coisa...  
[1906 ms] Mutex ocupado. Fazendo outra coisa...  
[2117 ms] Mutex ocupado. Fazendo outra coisa...  
[2328 ms] Mutex ocupado. Fazendo outra coisa...  
[2539 ms] Mutex ocupado. Fazendo outra coisa...  
[2750 ms] Mutex ocupado. Fazendo outra coisa...  
[2961 ms] Mutex ocupado. Fazendo outra coisa...  
[3172 ms] Mutex ocupado. Fazendo outra coisa...  
[3383 ms] Mutex ocupado. Fazendo outra coisa...  
[3594 ms] Mutex ocupado. Fazendo outra coisa...  
[3805 ms] Mutex ocupado. Fazendo outra coisa...  
[4016 ms] Mutex ocupado. Fazendo outra coisa...  
[4227 ms] Mutex ocupado. Fazendo outra coisa...  
[4438 ms] Mutex ocupado. Fazendo outra coisa...  
[4649 ms] Mutex ocupado. Fazendo outra coisa...  
[4860 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[5070 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[5280 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[5490 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
```

```
[5700 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[5910 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[6120 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[6330 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!
```

Utilizando timeout = 3

```
[3 ms] Mutex ocupado. Fazendo outra coisa...  
[207 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[415 ms] Mutex ocupado. Fazendo outra coisa...  
[622 ms] Mutex ocupado. Fazendo outra coisa...  
[829 ms] Mutex ocupado. Fazendo outra coisa...  
[1036 ms] Mutex ocupado. Fazendo outra coisa...  
[1243 ms] Mutex ocupado. Fazendo outra coisa...  
[1450 ms] Mutex ocupado. Fazendo outra coisa...  
[1657 ms] Mutex ocupado. Fazendo outra coisa...  
[1864 ms] Mutex ocupado. Fazendo outra coisa...  
[2071 ms] Mutex ocupado. Fazendo outra coisa...  
[2275 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[2483 ms] Mutex ocupado. Fazendo outra coisa...  
[2690 ms] Mutex ocupado. Fazendo outra coisa...  
[2897 ms] Mutex ocupado. Fazendo outra coisa...  
[3104 ms] Mutex ocupado. Fazendo outra coisa...  
[3311 ms] Mutex ocupado. Fazendo outra coisa...  
[3518 ms] Mutex ocupado. Fazendo outra coisa...  
[3725 ms] Mutex ocupado. Fazendo outra coisa...  
[3932 ms] Mutex ocupado. Fazendo outra coisa...  
[4139 ms] Mutex ocupado. Fazendo outra coisa...  
[4343 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[4551 ms] Mutex ocupado. Fazendo outra coisa...  
[4758 ms] Mutex ocupado. Fazendo outra coisa...  
[4965 ms] Mutex ocupado. Fazendo outra coisa...  
[5172 ms] Mutex ocupado. Fazendo outra coisa...  
[5379 ms] Mutex ocupado. Fazendo outra coisa...  
[5586 ms] Mutex ocupado. Fazendo outra coisa...  
[5793 ms] Mutex ocupado. Fazendo outra coisa...  
[6000 ms] Mutex ocupado. Fazendo outra coisa...  
[6207 ms] Mutex ocupado. Fazendo outra coisa...  
[6411 ms] Mutex adquirido. Fazendo uso do recurso compartilhado!  
[6619 ms] Mutex ocupado. Fazendo outra coisa...  
[6826 ms] Mutex ocupado. Fazendo outra coisa...
```

4. Análise dos resultados

Questão: Em uma aplicação de controle em tempo real, por que pode ser inadequado bloquear uma tarefa indefinidamente esperando por um recurso que não é essencial para sua operação principal?

Porque pode ocorrer uma situação de starvation e o sistema como um todo ser paralizado devido ao fato de essa tarefa estar presa aguardando o recurso não-essencial.

Atividade 8 — Mini Sistema Industrial

1. Objetivo

Integrar prioridades, semáforos e mutex em uma aplicação de tempo real que simula uma esteira industrial com detecção e processamento de peças, supervisão, registro de eventos (log) e tratamento de alarmes.

2. Diagrama da aplicação

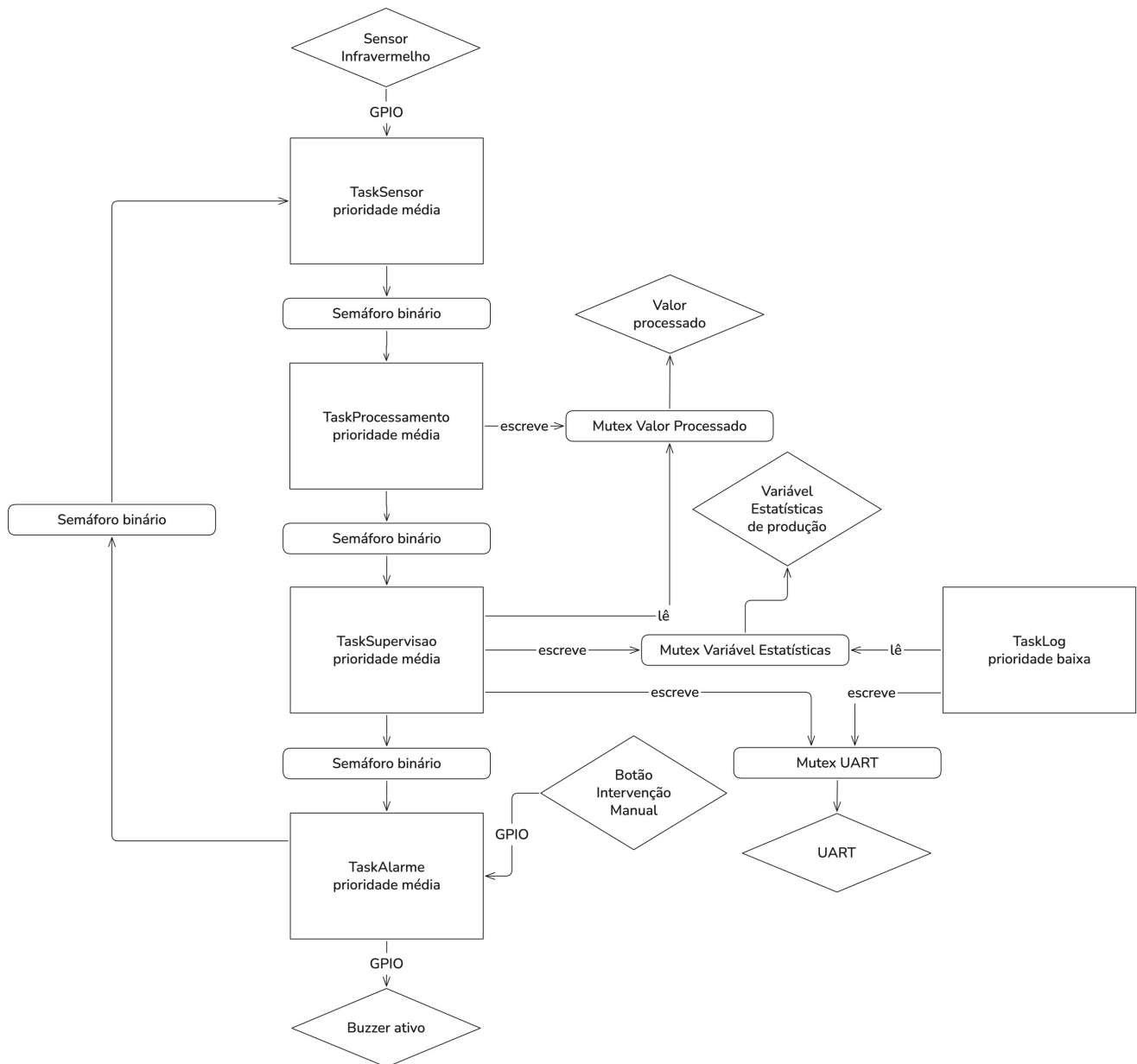


Diagrama da Atividade 8

3. Definição da arquitetura

Plataforma

Recurso	Pino	Configuração
Sensor infravermelho (IR_SENSOR_PIN)	PA5	Entrada, sem pull. Ativo em nível baixo (0 = peça detectada)
Buzzer ativo (BUZZER_PIN)	PA4	Saída. Nível alto liga o buzzer
Botão K1 (BTN_K1)	PE3	Entrada com pull-up. Pressionado = 0
USART1	PA9 (TX) / PA10 (RX)	115200 bps, 8N1. <code>printf</code> redirecionado por <code>write()</code>

Tarefas e prioridades

Tarefa	Prioridade	Bloqueia em
TaskSensor	osPriorityNormal	semaforoOperacao e osDelay
TaskProcessamento	osPriorityNormal	semaforoSensor
TaskSupervisao	osPriorityNormal	semaforoSupervisao
TaskLog	osPriorityLow	osDelay(10000)
TaskAlarme	osPriorityLow	semaforoAlarme e osDelay(50)

As três tarefas do fluxo da peça (sensor → processamento → supervisão) formam uma cascata: cada uma só executa quando a anterior libera o semáforo correspondente, de modo que a ordem de execução é determinada pela sinalização e não pela prioridade relativa. TaskLog não possui requisito rígido de tempo e TaskAlarme só precisa ser acordada em caso de falha; ambas ficam em prioridade baixa e executam sempre que as tarefas de prioridade normal estão bloqueadas.

Semáforos

Semáforo	Valor inicial	Função
semaforoSensor	0	TaskSensor sinaliza peça detectada para TaskProcessamento
semaforoSupervisao	0	TaskProcessamento sinaliza resultado pronto para TaskSupervisao
semaforoAlarme	0	TaskSupervisao sinaliza falha para TaskAlarme
semaforoOperacao	1	Permissão de operação da esteira. TaskAlarme o retém enquanto o alarme está ativo

Mutex

Mutex	Recurso protegido	Tarefas que o utilizam
mutexValorProcessado	variável valorProcessado	TaskProcessamento (escrita), TaskSupervisao (leitura)
mutexVariavelEstatisticas	contadores pecasOK e pecasFalha	TaskSupervisao (escrita), TaskLog (leitura)
mutexUart	transmissão pela USART1	TaskSupervisao, TaskLog

Informações apresentadas pela UART

- Uma linha por peça processada, com o valor calculado, ou uma mensagem de erro quando o valor é maior ou igual a 95 (aproximadamente 5% das peças, pois o valor é `rand() % 100`).
- Um sumário a cada 10 s com o total de peças processadas e de erros registrados.

4. Implementação e experimentos

Código das tarefas

```
/* USER CODE BEGIN PV */
int valorProcessado = 0;
int pecasOK = 0;
int pecasFalha = 0;
/* USER CODE END PV */
```

```
void TaskSensor_fun(void *argument)
{
    /* USER CODE BEGIN 5 */
    /* Infinite loop */
    for(;;)
    {
        osSemaphoreAcquire(semaforoOperacaoHandle, osWaitForever); // Permissão para operar

        if(HAL_GPIO_ReadPin(IR_SENSOR_PIN_GPIO_Port, IR_SENSOR_PIN_Pin) == GPIO_PIN_RESET){ // Sensor ativo em nível baixo
            osSemaphoreRelease(semaforoSensorHandle);
            osDelay(100);
            while(HAL_GPIO_ReadPin(IR_SENSOR_PIN_GPIO_Port, IR_SENSOR_PIN_Pin) == GPIO_PIN_RESET);
        }

        osSemaphoreRelease(semaforoOperacaoHandle);
        osDelay(10);
    }
    /* USER CODE END 5 */
}
```

```
void TaskProcessamento_fun(void *argument)
{
    /* USER CODE BEGIN TaskProcessamento_fun */
    /* Infinite loop */
    for(;;)
    {
        if(osSemaphoreAcquire(semaforoSensorHandle, osWaitForever) == osOK){

            osMutexAcquire(mutexValorProcessadoHandle, osWaitForever);
            valorProcessado = (rand() % 100);
            osMutexRelease(mutexValorProcessadoHandle);

            osSemaphoreRelease(semaforoSupervisaoHandle);
        }
    }
    /* USER CODE END TaskProcessamento_fun */
}
```

```
void TaskSupervisao_fun(void *argument)
{
    /* USER CODE BEGIN TaskSupervisao_fun */
    int valorProcessadoLocal = 0;
    /* Infinite loop */
    for(;;)
    {
        if(osSemaphoreAcquire(semaforoSupervisaoHandle, osWaitForever) == osOK){

            osMutexAcquire(mutexValorProcessadoHandle, osWaitForever);
            valorProcessadoLocal = valorProcessado;
            osMutexRelease(mutexValorProcessadoHandle);

            // Ordem de aquisição padronizada: UART primeiro, estatísticas depois
            osMutexAcquire(mutexUartHandle, osWaitForever);
            osMutexAcquire(mutexVariavelEstatisticasHandle, osWaitForever);

            if(valorProcessadoLocal >= 95){
                pecasFalha++;
                printf("[%lu ms] ERRO DETECTADO - Necessária intervenção manual\r\n", osKernelGetTickCount());
                osSemaphoreRelease(semaforoAlarmeHandle);
            } else {
                pecasOK++;
                printf("[%lu ms] Peça processada no valor de %d\r\n", osKernelGetTickCount(), valorProcessadoLocal);
            }

            osMutexRelease(mutexUartHandle);
            osMutexRelease(mutexVariavelEstatisticasHandle);
        }
    }
    /* USER CODE END TaskSupervisao_fun */
}
```

```

void TaskLog_fun(void *argument)
{
    /* USER CODE BEGIN TaskLog_fun */
    /* Infinite loop */
    for(;;)
    {
        // Mesma ordem de aquisição de TaskSupervisao
        osMutexAcquire(mutexUartHandle, osWaitForever);
        osMutexAcquire(mutexVariavelEstatisticasHandle, osWaitForever);
        printf("[%lu ms][LOG] PEÇAS PROCESSADAS: %d ERROS REGISTRADOS: %d\r\n", osKernelGetTickCount(), pecasOK, pecasFalha);
        osMutexRelease(mutexUartHandle);
        osMutexRelease(mutexVariavelEstatisticasHandle);
        osDelay(10000);
    }
    /* USER CODE END TaskLog_fun */
}

```

```

void TaskAlarme_fun(void *argument)
{
    /* USER CODE BEGIN TaskAlarme_fun */
    HAL_GPIO_WritePin(BUZZER_PIN_GPIO_Port, BUZZER_PIN_Pin, 0);
    /* Infinite loop */
    for(;;)
    {
        if(osSemaphoreAcquire(semaforoAlarmeHandle, osWaitForever) == osOK){
            osSemaphoreAcquire(semaforoOperacaoHandle, osWaitForever); // Retém a permissão de operação
            HAL_GPIO_WritePin(BUZZER_PIN_GPIO_Port, BUZZER_PIN_Pin, 1);

            while(HAL_GPIO_ReadPin(BTN_K1_GPIO_Port, BTN_K1_Pin) == 1){
                osDelay(50);
            }

            HAL_GPIO_WritePin(BUZZER_PIN_GPIO_Port, BUZZER_PIN_Pin, 0);
            osSemaphoreRelease(semaforoOperacaoHandle); // Retoma a operação
        }
    }
    /* USER CODE END TaskAlarme_fun */
}

```

Funcionamento normal

Saída registrada:

```

[0 ms][LOG] PEÇAS PROCESSADAS: 0 ERROS REGISTRADOS: 0
[3190 ms] Peça processada no valor de 0
[4000 ms] Peça processada no valor de 33
[4330 ms] Peça processada no valor de 43
[4580 ms] Peça processada no valor de 62
[4810 ms] Peça processada no valor de 29
[8350 ms] Peça processada no valor de 0
[9510 ms] Peça processada no valor de 8
[9880 ms] Peça processada no valor de 52
[10004 ms][LOG] PEÇAS PROCESSADAS: 8 ERROS REGISTRADOS: 0
[11880 ms] Peça processada no valor de 56
[12160 ms] Peça processada no valor de 56
[12570 ms] Peça processada no valor de 19
[12780 ms] Peça processada no valor de 11
[13140 ms] Peça processada no valor de 51
[13430 ms] Peça processada no valor de 43
[13660 ms] Peça processada no valor de 5
[13910 ms] Peça processada no valor de 8
[14170 ms] Peça processada no valor de 93
[14360 ms] Peça processada no valor de 30
[14570 ms] Peça processada no valor de 66
[14750 ms] Peça processada no valor de 69
[14950 ms] Peça processada no valor de 32
[15070 ms] Peça processada no valor de 17
[15230 ms] Peça processada no valor de 47
[15360 ms] Peça processada no valor de 72
[15500 ms] Peça processada no valor de 68
[15610 ms] Peça processada no valor de 80
[15750 ms] Peça processada no valor de 23
[15880 ms] Peça processada no valor de 49
[16000 ms] Peça processada no valor de 92
[16120 ms] Peça processada no valor de 64
[16250 ms] Peça processada no valor de 69
[16476 ms] Peça processada no valor de 51
[16596 ms] Peça processada no valor de 27
[16726 ms] Peça processada no valor de 90
[16836 ms] Peça processada no valor de 24
[16966 ms] Peça processada no valor de 35
[17186 ms] Peça processada no valor de 20
[17296 ms] Peça processada no valor de 44
[17426 ms] Peça processada no valor de 10
[17566 ms] Peça processada no valor de 62
[17696 ms] Peça processada no valor de 84
[17856 ms] Peça processada no valor de 63
[17976 ms] Peça processada no valor de 1
[18086 ms] Peça processada no valor de 10
[18326 ms] Peça processada no valor de 36
[18566 ms] Peça processada no valor de 76
[18716 ms] Peça processada no valor de 31
[18943 ms] Peça processada no valor de 29
[19063 ms] ERRO DETECTADO - Necessária intervenção manual
[20009 ms][LOG] PEÇAS PROCESSADAS: 48 ERROS REGISTRADOS: 1

```

```
[30014 ms][LOG] PEÇAS PROCESSADAS: 48 ERROS REGISTRADOS: 1
[34623 ms] Peça processada no valor de 75
[35314 ms] Peça processada no valor de 91
[37054 ms] Peça processada no valor de 90
[38164 ms] Peça processada no valor de 44
[38484 ms] Peça processada no valor de 34
[38904 ms] Peça processada no valor de 25
[39294 ms] Peça processada no valor de 29
[40019 ms][LOG] PEÇAS PROCESSADAS: 55 ERROS REGISTRADOS: 1
[42124 ms] Peça processada no valor de 30
[42554 ms] Peça processada no valor de 27
[42954 ms] Peça processada no valor de 26
[43534 ms] Peça processada no valor de 43
[43774 ms] Peça processada no valor de 34
[44324 ms] Peça processada no valor de 4
[46504 ms] Peça processada no valor de 60
[46644 ms] Peça processada no valor de 49
[46784 ms] Peça processada no valor de 20
[47337 ms] Peça processada no valor de 56
[47487 ms] Peça processada no valor de 32
[50024 ms][LOG] PEÇAS PROCESSADAS: 66 ERROS REGISTRADOS: 1
```

Descrição: a cada peça detectada pelo sensor, `TaskSensor` libera `semaforoSensor`, `TaskProcessamento` calcula o valor e libera `semaforoSupervisao`, e `TaskSupervisao` imprime o resultado. `TaskLog` imprime o sumário imediatamente na inicialização e depois a cada 10 s. Quando o valor sorteado é maior ou igual a 95, `TaskSupervisao` registra o erro e libera `semaforoAlarme`; `TaskAlarme` então retém `semaforoOperacao`, aciona o buzzer e aguarda o botão K1. Durante esse intervalo `TaskSensor` permanece em estado Blocked e nenhuma nova peça é aceita.

Desafio: Possíveis problemas de concorrência

Problema 1 — Deadlock entre mutex

Na versão com o problema, `TaskSupervisao` e `TaskLog` adquirem os mutex em ordens opostas:

```
// TaskSupervisao (versão com o problema)
osMutexAcquire(mutexVariavelEstatisticasHandle, osWaitForever);
osMutexAcquire(mutexUartHandle, osWaitForever);
```

```
// TaskLog (versão com o problema)
osMutexAcquire(mutexUartHandle, osWaitForever);
osMutexAcquire(mutexVariavelEstatisticasHandle, osWaitForever);
```

Se `TaskLog` adquire `mutexUart` e é preemptada por `TaskSupervisao` antes de adquirir `mutexVariavelEstatisticas`, `TaskSupervisao` obtém `mutexVariavelEstatisticas` e passa a esperar `mutexUart`. `TaskLog` retoma, passa a esperar `mutexVariavelEstatisticas`, e as duas ficam em Blocked indefinidamente, cada uma aguardando o mutex retido pela outra.

Solução: padronizar a ordem de aquisição. As duas tarefas passam a adquirir sempre `mutexUart` primeiro e `mutexVariavelEstatisticas` depois, conforme o código apresentado acima.

Problema 2 — Esteira não para após o alarme

Na versão com o problema, `TaskSensor` não adquiria `semaforoOperacao`. `TaskAlarme` acionava o buzzer, mas `TaskSensor` continuava lendo o pino e liberando `semaforoSensor`, de modo que novas peças continuavam entrando no pipeline durante o alarme.

```
// TaskSensor (versão com o problema)
for(;;)
{
    if(HAL_GPIO_ReadPin(IR_SENSOR_PIN_GPIO_Port, IR_SENSOR_PIN_Pin) == GPIO_PIN_RESET){
        osSemaphoreRelease(semaforoSensorHandle);
        osDelay(100);
        while(HAL_GPIO_ReadPin(IR_SENSOR_PIN_GPIO_Port, IR_SENSOR_PIN_Pin) == GPIO_PIN_RESET);
    }
    osDelay(10);
}
```

Solução: o semáforo binário `semaforoOperacao`, inicializado em 1, funciona como permissão de operação. `TaskSensor` o adquire antes de ler o pino e o libera ao final de cada ciclo. Quando há erro, `TaskAlarme` adquire esse semáforo e não o libera até o botão K1 ser pressionado, o que mantém `TaskSensor` em Blocked durante o alarme.

5. Análise dos resultados

Requisitos e mecanismos

Requisito	Mecanismo	Justificativa
Definir as prioridades das tarefas	Prioridade	Fluxo da peça em <code>osPriorityNormal</code> ; <code>TaskLog</code> em <code>osPriorityLow</code> , pois não têm requisito rígido de tempo e executam quando as demais estão bloqueadas
Acordar a próxima etapa quando há dado ou evento	Semáforo binário	<code>semaforoSensor</code> , <code>semaforoSupervisao</code> e <code>semaforoAlarme</code> mantêm as tarefas em <code>Blocked</code> até serem sinalizadas, sem polling. Quem libera não é quem adquire
Permitir ou interromper a operação da esteira	Semáforo binário	<code>semaforoOperacao</code> autoriza <code>TaskSensor</code> a operar; <code>TaskAlarme</code> o retém para parar a esteira e o libera ao fim do alarme
Proteger variável compartilhada de resultado	Mutex	<code>mutexValorProcessado</code> garante acesso exclusivo a <code>valorProcessado</code> entre <code>TaskProcessamento</code> e <code>TaskSupervisao</code>
Proteger contadores de estatística	Mutex	<code>mutexVariavelEstatisticas</code> evita leitura inconsistente de <code>pecasOK</code> e <code>pecasFalha</code> por <code>TaskLog</code> durante a atualização por <code>TaskSupervisao</code>
Controlar acesso à UART	Mutex	<code>mutexUart</code> mantém a integridade das mensagens de <code>TaskSupervisao</code> e <code>TaskLog</code>
Evitar deadlock	Ordem de aquisição	Aquisição sempre na ordem <code>mutexUart</code> → <code>mutexVariavelEstatisticas</code> nas duas tarefas
Tarefas que podem ficar bloqueadas	Semáforo e <code>osDelay</code>	Todas as tarefas bloqueiam em semáforo ou <code>osDelay</code> , com a exceção da espera ativa em <code>TaskSensor</code>
Informações pela UART	Mutex	Uma linha por peça processada ou erro e um sumário a cada 10 s

Análise final

Tabela de mecanismos

Problema	Prioridade	Semáforo	Mutex
Definir qual tarefa deve executar primeiro	X		
Esperar pela ocorrência de um evento		X	
Proteger uma variável compartilhada			X
Controlar acesso exclusivo à UART			X
Representar três recursos disponíveis		X	
Atender rapidamente uma tarefa crítica	X		
Sincronizar sensor e processamento		X	

Questão final

Questão: Diante de um problema em uma aplicação FreeRTOS, como decidir se a solução exige alterar a prioridade de uma tarefa, utilizar um semáforo ou proteger um recurso utilizando mutex?

A decisão depende da natureza do problema. Caso seja um problema de urgência ou latência, deve-se alterar as prioridades de cada tarefa. Caso seja de sincronização no tempo, utiliza-se semáforos. E caso seja um problema de garantir a integridade de um recurso crítico, utiliza-se mutexes.

Diferença entre prioridade, semáforo e mutex

1. **Prioridade:** Define a ordem em que as tarefas em Ready recebem o processador. Exemplo: Tarefas que exigem baixa latência como controle de motores em um sistema industrial terão prioridade alta, enquanto tarefas menos essenciais, como as responsáveis por interfaces gráficas ou sistemas de log ficam com prioridades baixas para evitar tomar o tempo em processamento de tarefas que, caso falhem, possam causar prejuízos significativos.
2. **Semáforo:** Autoriza uma tarefa a prosseguir quando um evento ocorre ou quando há unidades disponíveis de um recurso. Exemplo: Para evitar o gasto desnecessário de CPU por polling, uma tarefa que consome um dado fornecido por outra tarefa responsável por processar dados de um ADC permanece bloqueada até esse dado estar pronto.
3. **Mutex:** concede a uma tarefa por vez o uso de um recurso compartilhado. Exemplo: um barramento I2C usado por tarefas que leem sensores diferentes, onde cada transação completa precisa ocorrer sem interferência de outra tarefa.